

Caso: Modelagem de Tópicos com o Algoritmo LDA na sklearn

Aula 11 – Inteligência Computacional

Eduardo Corrêa Gonçalves

25/09/2026

Sumário

Introdução

Passos para criar um modelo LDA na sklearn

Exemplo Prático – corpus da aula 10 (alimentação e animais)

Passo a passo para a geração do modelo

Exercício – LDA em um corpus de resumos de filmes

Link para código com o exemplo da aula

- **Caso – Modelagem de Tópicos na sklearn (exemplos dessa aula)**
 - <https://colab.research.google.com/drive/10gUhOTVZAeMY-PylWTaREjSleMDeHxlg?usp=sharing>

Introdução (1/2)

Os 4 passos para criar um modelo LDA na sklearn:

1. Importação do corpus (conjunto de textos)
 2. Pré-processamento dos documentos
 3. Geração do modelo LDA.
 4. Visualização do modelo gerado (palavras de cada tópico + tópicos de cada documento)
- Não é um esquema fixo! Dependendo do problema, você pode alterar esse fluxo.
 - Por exemplo, muitas vezes é necessário utilizar um método para sugerir o valor de k (exemplo: coerência vetorial).
 - **Obs.:** Estamos apresentando os passos para criar o modelo. Mas na prática, também é necessário avaliar de forma intrínseca ou extrínseca a sua qualidade (usando medidas como a Perplexidade)

Introdução (2/2)

Vamos demonstrar cada etapa utilizando o mesmo corpus exemplo da aula passada:

Um corpus com 5 textos, em que alguns são predominantemente sobre “**alimentação**” e outros sobre “**animais**”

1	eu trouxe uma barra de chocolate para lanchar
2	eu gosto de comer chocolate, amendoim e castanhas
3	meu vizinho tem 2 cachorros
4	cachorros e gatos são inimigos mortais
5	cachorros não podem comer chocolate

LDA na sklearn: passo a passo (1/17)

PASSO 1: importação da base de dados (BD) rotulada

- Como o exemplo é pequeno, vamos definir o corpus em uma lista diretamente no código

```
corpus = [  
    "eu trouxe uma barra de chocolate para lanchar",  
    "eu gosto de comer chocolate, amendoim e castanhas",  
    "meu vizinho tem 2 cachorros",  
    "cachorros e gatos são inimigos mortais",  
    "cachorros não podem comer chocolate"  
]
```

LDA na sklearn: passo a passo (2/17)

Resultado ao final do PASSO 1

Corpus importado em memória.

```
['eu trouxe uma barra de chocolate para lanchar',  
'eu gosto de comer chocolate, amendoim e castanhas',  
'meu vizinho tem 2 cachorros',  
'cachorros e gatos são inimigos mortais',  
'cachorros não podem comer chocolate']
```

LDA na sklearn: passo a passo (3/17)

PASSO 2: pré-processamento

- No caso do LDA, o pré-processamento **obrigatório** consiste em converter o texto para uma matriz **bag of words** (BoW).
- Matriz em que:
 - Cada **linha** i corresponde ao **documento** i .
 - Cada **coluna** j corresponde a um **termo** do vocabulário.
 - Cada **célula** i, j registra a frequência do termo j no documento i .
- No pré-processamento, podemos também fazer outras operações, como:
 - remover stop words (palavras irrelevantes)
 - **lematizar** os termos
 - Ex.: "comendo" -> "comer", "cachorros" -> "cachorro"

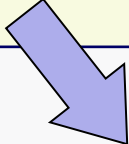
LDA na sklearn: passo a passo (4/17)

PASSO 2: pré-processamento

- Abaixo a BoW referente ao nosso corpus
- **Obs.:** palavras stop words foram excluídas: “de”, “uma”, “e”, etc.

corpus

"eu trouxe uma barra de chocolate para lanchar",
"eu gosto de comer chocolate, amendoim e castanhas",
"meu vizinho tem 2 cachorros",
"cachorros e gatos são inimigos mortais",
"cachorros não podem comer chocolate"



BoW

	amendoim	barra	cachorros	castanhas	chocolate	comer	gatos	gosto	inimigos	lanchar	mortais	trouxe	vizinho
0	0	1	0	0	1	0	0	0	0	1	0	1	0
1	1	0	0	1	1	1	0	1	0	0	0	0	0
2	0	0	1	0	0	0	0	0	0	0	0	0	1
3	0	0	1	0	0	0	1	0	1	0	1	0	0
4	0	0	1	0	1	1	0	0	0	0	0	0	0

LDA na sklearn: passo a passo (5/17)

PASSO 2: pré-processamento – geração da BoW

- A BoW é gerada com a classe **CountVectorizer** do módulo **sklearn.feature_extraction.text**.
- Aplica-se a função **fit_transform**. Como resultado, obtemos:
 - Uma BoW no formato matriz numpy (formato puramente numérico)
 - Uma lista com os tokens (termos) encontrados.

```
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer

# lista com algumas stopwords em português
# (serão descartadas)
stoplist_pt = ['eu', 'uma', 'para', 'de', 'e', 'tem',
               'são', 'não', 'podem', 'meu', '2', 'com']

# geração da matriz bag of words
c_vect = CountVectorizer(stop_words=stoplist_pt)
bow = c_vect.fit_transform(corpus).toarray()

# pega a lista de tokens (termos)
tokens = c_vect.get_feature_names_out()
```

LDA na sklearn: passo a passo (6/17)

Resultado ao final do PASSO 2

- BoW gerada! Nosso corpus está convertido.

```
# imprime a matriz no formato original
print('BoW gerada (formato original):')
print(bow); print()

# lista de tokens (termos)
print('Tokens:'); print(tokens)
```



BoW gerada (formato original):

```
[[0 1 0 0 1 0 0 0 0 1 0 1 0]
 [1 0 0 1 1 1 0 1 0 0 0 0 0]
 [0 0 1 0 0 0 0 0 0 0 0 0 1]
 [0 0 1 0 0 0 1 0 1 0 1 0 0]
 [0 0 1 0 1 1 0 0 0 0 0 0 0]]
```

Tokens:

```
['amendoim' 'barra' 'cachorros' 'castanhas' 'chocolate' 'comer' 'gatos'
 'gosto' 'inimigos' 'lanchar' 'mortais' 'trouxe' 'vizinho']
```

LDA na sklearn: passo a passo (7/17)

Obs.: podemos converter a BoW de uma array numpy para um DataFrame

- Assim a visualização fica mais agradável.

```
# gerando um DataFrame com a BoW  
df_bow = pd.DataFrame(bow, columns=tokens)  
df_bow
```



	amendoim	barra	cachorros	castanhas	chocolate	comer	gatos	gosto	inimigos	lanchar	mortais	trouxe	vizinho
0	0	1	0	0	1	0	0	0	0	1	0	1	0
1	1	0	0	1	1	1	0	1	0	0	0	0	0
2	0	0	1	0	0	0	0	0	0	0	0	0	1
3	0	0	1	0	0	0	1	0	1	0	1	0	0
4	0	0	1	0	1	1	0	0	0	0	0	0	0

LDA na sklearn: passo a passo (8/17)

PASSO 3: geração do modelo LDA

- (1). **importar o pacote** com o algoritmo, nesse caso **LatentDirichletAllocation** (módulo **decomposition** da **sklearn**)
- (2). **Instanciar um objeto** da classe **LatentDirichletAllocation**, indicando os valores de hiperparâmetros que você deseja.
 - Utilizaremos quase todos os hiperparâmetros com valor default, exceto *k* (que é chamado **n_components**) e **random_state**.
- (3). Usar o método **fit_transform**, para fazer **gerar o modelo** a partir da BoW.

```
from sklearn.decomposition import LatentDirichletAllocation

# geração do modelo LDA com 2 tópicos (n_components = 2)
lda = LatentDirichletAllocation(n_components=2,
                                random_state=0)
modelo = lda.fit_transform(bow)
```

LDA na sklearn: passo a passo (10/17)

PASSO 3: geração do modelo LDA

- Abaixo a lista de hiperparâmetros do algoritmo. Destacamos alguns abaixo:
 - `n_components` é o `k`
 - `doc_topic_prior` é o `alpha`. Valor default $1/n_components$
 - `doc_word_prior` é o `beta`. Valor default $1/n_components$
 - `max_iter` é o número máximo de iterações, cujo valor default é 10.

LatentDirichletAllocation

```
class sklearn.decomposition.LatentDirichletAllocation(n_components=10,  
*, doc_topic_prior=None, topic_word_prior=None, learning_method='batch',  
learning_decay=0.7, learning_offset=10.0, max_iter=10, batch_size=128,  
evaluate_every=-1, total_samples=10000000.0, perp_tol=0.1,  
mean_change_tol=0.001, max_doc_update_iter=100, n_jobs=None, verbose=0,  
random_state=None)
```

[\[source\]](#)

LDA na sklearn: passo a passo (11/17)

Resultado ao final do PASSO 3

- Após o `fit_transform*`, dois arrays de resultados serão gerados:
 - (1) `lda.components_`: array $k \times d$ com os pesos (pseudocontagens**) de cada palavra em cada tópico. k é o número de tópicos e d o número de termos

```
print(lda.components_)
```

```
[[0.50570962 0.50723479 3.39107491 0.50570962 0.54200772 0.56365645  
 1.49602327 0.50570962 1.49602327 0.50723479 1.49602327 0.50723479  
 1.49199975]  
 [1.49429038 1.49276521 0.60892509 1.49429038 3.45799228 2.43634355  
 0.50397673 1.49429038 0.50397673 1.49276521 0.50397673 1.49276521  
 0.50800025]]
```

- **Interpretação:**
 - Para o **tópico 0** (1ª linha da matriz), o 3º termo tem maior peso (“**cachorros**”)
 - Para o **tópico 1** (2ª linha), o 5º termo tem maior peso (“**chocolate**”)

* Dessa vez não é “fit”, mas “fit_transform”. Utilizei o Gemini para auxiliar na geração do código.

** O manual da sklearn usa essa palavra, “pseudocontagens”

LDA na sklearn: passo a passo (12/17)

Resultado ao final do PASSO 3

- (2) **modelo** gerado pelo **fit_transform**, que é uma matriz com o percentual de cada tópico em cada documento.
 - Possui dimensão $N \times k$.
 - N é o número de documentos; k é o número de tópicos.

```
print(modelo)
```

```
[[0.1049356  0.8950644 ]  
 [0.08723161 0.91276839]  
 [0.82921649 0.17078351]  
 [0.89717495 0.10282505]  
 [0.37330952 0.62669048]]
```

- **Interpretação:**
 - **Doc 0:** 10,5% tópico 0 e 89,5% tópico 1 (1ª linha da matriz)
 - **Doc 1:** 8,7% tópico 0 e 91,3% tópico 1 (2ª linha da matriz)
 - **Doc 2:** 82,9% tópico 0 e 17,1% tópico 1 (3ª linha da matriz)
 - **Doc 3:** 89,7% tópico 0 e 10,3% tópico 1 (4ª linha da matriz)
 - **Doc 4:** 37,3% tópico 0 e 62,7% tópico 1 (5ª linha da matriz)

LDA na sklearn: passo a passo (13/17)

PASSO 4: Visualização do Modelo

- Agora temos que explorar as duas matrizes geradas (**components_** e **modelo**) para visualizar as 2 partes que compõem a saída do modelo LDA:
 - PARTE 1:
 - **tópicos identificados** → está em **components_**
 - PARTE 2:
 - **% de cada tópico em cada documento** → está em **modelo**
- Assim será possível interpretar os resultados...

LDA na sklearn: passo a passo (13/17)

PASSO 4: Visualização do Modelo – Parte 1: Tópicos Identificados

- Como sabemos, os tópicos são descritos por suas palavras mais relevantes. Abaixo um código para extraí-las.

[illegible]

LDA na sklearn: passo a passo (14/17)

PASSO 4: Visualização do Modelo – Parte 1: Tópicos Identificados

```
print(mat_topics)
```

```
[[ 3.74166191  3.75294639 25.09000265  3.74166191  4.01022552  4.17040092
 11.06882884  3.74166191 11.06882884  3.75294639 11.06882884  3.75294639
 11.0390595 ]
 [ 8.54644115  8.53771809  3.48268486  8.54644115 19.77763357 13.93441802
 2.88244344  8.54644115  2.88244344  8.53771809  2.88244344  8.53771809
 2.90545553]]
```

Tópicos no formato array

df_topics

	Tópico 0	Tópico 1
amendoim	3.741662	8.546441
barra	3.752946	8.537718
cachorros	25.090003	3.482685
castanhas	3.741662	8.546441
chocolate	4.010226	19.777634
comer	4.170401	13.934418
gatos	11.068829	2.882443
gosto	3.741662	8.546441
inimigos	11.068829	2.882443
lanchar	3.752946	8.537718
mortais	11.068829	2.882443
trouxe	3.752946	8.537718
vizinho	11.039060	2.905456

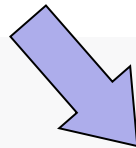
Tópicos em um
DataFrame

LDA na sklearn: passo a passo (15/17)

PASSO 4: Visualização do Modelo – Parte 1: Tópicos Identificados

- Podemos imprimir cada coluna do DataFrame em separado, ordenando decendente pelo valor para pegar as 3 (ou mais) palavras principais.

```
print('3 palavras mais importantes por tópico:')
k = 2
for i in range(k):
    nome_col = 'Topico_' + str(i)
    topico_i = df_topicos[nome_col].sort_values(ascending=False)
    print('\n\n3 palavras mais relevantes do Tópico', i)
    print(topico_i[:3])
```



```
3 palavras mais relevantes do Tópico 0
cachorros      25.090003
mortais        11.068829
gatos          11.068829
Name: Topico_0, dtype: float64
```

```
3 palavras mais relevantes do Tópico 1
chocolate      19.777634
comer          13.934418
castanhas      8.546441
Name: Topico_1, dtype: float64
```

LDA na sklearn: passo a passo (16/17)

PASSO 4: Visualização do Modelo – Parte 2: % de cada tópico em cada documento

- Para a visualizar a parte 2 da saída, vamos fazer um loop na matriz **modelo**.

```
## visualizando a PARTE 2 da saída
# percentual de cada tópico em cada documento

# basta fazer um loop na estrutura "modelo"
for i, probs in enumerate(modelo):
    topico_dominante = np.argmax(probs) # vai retornar 0 ou 1
    print(f'Doc.{i}: "{corpus[i]}"')
    print(f'Tópico 0: {probs[0] * 100:.2f}% - Tópico 1: {probs[1] * 100:.2f}%')
    print(f'Tópico dominante: {topico_dominante}')
    print()
```

LDA na sklearn: passo a passo (17/17)

PASSO 4: Visualização do Modelo – Parte 2: % de cada tópico em cada documento

Doc.0: "eu trouxe uma barra de chocolate para lanchar"

Tópico 0: 10.49% - Tópico 1: 89.51%

Tópico dominante: 1

Doc.1: "eu gosto de comer chocolate, amendoim e castanhas"

Tópico 0: 8.72% - Tópico 1: 91.28%

Tópico dominante: 1

Doc.2: "meu vizinho tem 2 cachorros"

Tópico 0: 82.92% - Tópico 1: 17.08%

Tópico dominante: 0

Doc.3: "cachorros e gatos são inimigos mortais"

Tópico 0: 89.72% - Tópico 1: 10.28%

Tópico dominante: 0

Doc.4: "cachorros não podem comer chocolate"

Tópico 0: 37.33% - Tópico 1: 62.67%

Tópico dominante: 1

EXERCÍCIO

- Seja um corpus contendo o texto com o resumo de 110 filmes:
 - <https://raw.githubusercontent.com/edubd/bd/refs/heads/main/resumos110.txt>
 - Nesse corpus:
 - Os 50 primeiros filmes são romances
 - Os 50 seguintes terror
 - Os 10 últimos comédia
- Gere e analise um modelo LDA para esse corpus fazendo $k = 3$.

Referências

- LatentDirichlet Allocation. *In*: scikit-learn documentation. Disponível em < <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.LatentDirichletAllocation.html> >. Acesso em: 11 set. 2026.
- Latent Dirichlet Allocation (LDA). *In*: scikit-learn documentation. Disponível em < <https://scikit-learn.org/stable/modules/decomposition.html#latentdirichletallocation> >. Acesso em: 11 set. 2026.
- NAIR, G. **Text Mining 101: Topic Modeling**. [S.l.]: KDNuggets, 2016. Disponível em < <https://www.kdnuggets.com/2016/07/text-mining-101-topic-modeling.html> >. Acesso em: 11 set. 2026.
- SERRANO, L. **Latent Dirichlet Allocation (Part 1 of 2)**. [S.l.]: YouTube, 2020. 1 video (26 min. 55 seg.). Disponível em < <https://www.youtube.com/watch?v=T05t-SqKArY> >. Acesso em: 03 set. 2026.
- SERRANO, L. **Training Latent Dirichlet Allocation: Gibbs Sampling (Part 2 of 2)**. [S.l.]: YouTube, 2020. 1 video (26 min. 30 seg.). Disponível em < https://www.youtube.com/watch?v=BaM1uiCpj_E >. Acesso em: 03 set. 2026.